



SECURITY REVIEW • PREPARED FOR

letscash

Launchpad on Uniswap v4 with built-in
fee and burn mechanics for Robinhood
Chain

DATE

02.08.2026

COMMIT

98a92370

REMEDICATION

4e732684

Phase & process

1. About SBSecurity

SBSecurity is a team of skilled smart contract security researchers. Based on the audits conducted and numerous vulnerabilities reported, we strive to provide the absolute best security service and client satisfaction. While it's understood that 100% security and bug-free code cannot be guaranteed by anyone, we are committed to giving our utmost to provide the best possible outcome for you and your product.

Book a Security Review with us at sbsecurity.net or reach out on Telegram [@blckhv](https://t.me/blckhv).

2. Disclaimer

A smart contract security review can only show the presence of vulnerabilities **but not their absence**. Audits are a time, resource, and expertise-bound effort where skilled technicians evaluate the codebase and their dependencies using various techniques to find as many flaws as possible and suggest security related improvements. We as a company stand behind our brand and the level of service that is provided but also recommend subsequent security reviews, on-chain monitoring, and high whitehat incentivization.

3. Risk classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

3.1 IMPACT

- **High** - leads to a significant loss of assets in the protocol or significantly harms a group of users.
- **Medium** - leads to a moderate loss of assets in the protocol or some disruption of the protocol's functionality.
- **Low** - funds are not at risk.

3.2 LIKELIHOOD

- **High** - almost certain to happen, easy to perform, or highly incentivized.
- **Medium** - only conditionally possible, but still relatively likely.
- **Low** - requires specific state or little-to-no incentive.

02 EXECUTIVE SUMMARY

Key findings

TOTAL FINDINGS

10

RESOLVED

10 of 10 · 100%

HIGH SEVERITY FINDINGS

0 none found

SEVERITY DISTRIBUTION

☒ Fixed & verified ☒ Acknowledged

CRITICAL 0

HIGH 0

MEDIUM 1

LOW 6

INFORMATIONAL 1

GAS OPT. 2

CLIENT

letscash

REPOSITORY

Private

METHODS

Manual review

CONTRACTS IN SCOPE

CashCatBuybackBurner.sol

CashCatHook.sol

CashCatToken.sol

LiquidityAmounts.sol

CashCatFactory.sol

CashCatSelfBurner.sol

CurrencySettler.sol

TIMELINE · AUGUST 2026

Audit · 9 days

Remediation · 2 days

KICK-OFF

20.07.2026

END OF AUDIT

28.07.2026

REMEDIATIONS START

01.08.2026

REMEDIATIONS END

02.08.2026

STARTING COMMIT HASH

98a923708dca83b06a1f90a25d8351c9015d4f95

FINAL COMMIT HASH

4e732684fb64914e8e2b2a4b0aab3b62f8c2bd0c

03 PROJECT SUMMARY

What letscash is

letscash is a memecoin launchpad on Uniswap v4. A single transaction deploys an immutable, ownerless token, creates its pool, and wires in a custom hook that takes a trading fee with a decaying launch premium. The creator's share of that fee stream is routed to a reward contract chosen at launch: buyback-and-burn of the token itself, or the platform's own buyback burner.

SECURITY POSTURE

BEFORE REMEDIATION

98A92370

AFTER REMEDIATION

4E732684

Fee accounting

M-01 · L-01

Exact-output swaps were charged on a smaller fee base than exact-input, bypassing the launch premium, and the public tax-rate views understated the fee actually taken during the decay window.

Self-burn safety

L-02 · L-03 · L-04 · G-02

The self-burner did not bind its registration to the pool it served, its burn path lacked reentrancy protection, the caller bounty was computed on requested rather than spent ETH, and a zero-output check ran only after the swap had done its work.

Configuration and deployment

L-05 · L-06 · I-01 · G-01

An ambient .env flag could disable the deployer-admin check, a creator could point the fee stream at an address that cannot receive ETH, and config setters accepted a missing config or a decay value that wrote dead state.

The team resolved all 10 findings presented in this report at the remediation commit; across the full engagement, 21 of 25 findings were resolved. The codebase changed substantially during remediation; only diffs related to the reported findings were reviewed, and unrelated additions remain out of scope.

Fifteen findings are withheld from this version of the report at the client's request; the Post-Audit Statement records their count, severity and status.

04 RECOMMENDATIONS

Beyond this audit

Measures that extend the protection of this review after launch, independent of any individual finding.

R-01

Monitor reward contracts in production

Alert on failed or stalled self-burn and buyback calls and on creator fee streams that stop clearing, so liveness edge cases are caught early.

R-02

Publish safe bounds for launch parameters

Document the fee, decay and per-call cap values a launch config may take, so launchers and integrators can check a configuration before relying on it.

06 CONTRACTS & ROLES

Core contracts

CONTRACT	ROLE
CashCatFactory.sol	One-transaction launches: deploys the token, creates the pool, and wires the hook and reward contracts.
CashCatHook.sol	Fee engine for letscash pools; takes the trading fee with a decaying launch premium and forwards the creator share.
CashCatToken.sol	The launched memecoin. Immutable and ownerless after deployment.
CashCatBuybackBurner.sol	Destination of platform revenue; splits it into a CASHCAT buyback-and-burn pot and a team pot.
CashCatSelfBurner.sol	Fee recipient for self-burn launches; buys the pool's own token through the pool and burns it.
CurrencySettler.sol	Library for settling and taking open currency deltas against the Uniswap v4 PoolManager.
LiquidityAmounts.sol	Math library for converting between token amounts and liquidity across price ranges.

Privileged roles

Protocol owner.

Controls the factory and hook configuration: launch configs, fee parameters, and the reward contracts a launch may select.

Token creator (launcher).

Chooses the launch config and reward mode at launch, and receives the creator share of pool fees.

Burn caller.

Anyone who triggers a self-burn or buyback purchase; paid a bounty out of the ETH the burn consumes.

07 FINDINGS

What we found

ID	TITLE	SEVERITY	STATUS
M-01	Exact-output swaps are charged on a smaller fee base than exact-input, bypassing the launch-window anti-snipe premium	MEDIUM	● FIXED
L-01	<code>buyTaxRate()</code> and <code>sellTaxRate()</code> understate the actual fee during the launch decay window	LOW	● FIXED
L-02	<code>CashCatSelfBurner.register</code> does not assert <code>key.toId() == poolId</code>	LOW	● FIXED
L-03	<code>CashCatSelfBurner.burn</code> is missing <code>nonReentrant</code>	LOW	● FIXED
L-04	Bounty in <code>burn</code> is calculated on requested input instead of ETH actually spent	LOW	● FIXED
L-05	<code>ALLOW_DEPLOYER_ADMIN</code> is read from ambient <code>.env</code> state and disables the only check that <code>OWNER</code> is not the deployer	LOW	● FIXED
L-06	<code>updateCreator</code> accepts an address that cannot receive ETH, permanently locking the pool fee stream	LOW	● FIXED
I-01	<code>setLaunchConfigEnabled</code> is missing a config existence check	INFORMATIONAL	● FIXED
G-01	<code>launchFeeDecay</code> can be non-zero while <code>baseFeeRate == launchFeeRate</code> writing dead state	GAS OPT.	● FIXED
G-02	<code>tokensOut == 0</code> revert runs after the swap already did wasted work in the callback	GAS OPT.	● FIXED

● MEDIUM SEVERITY · 1 FINDING

M-01

MEDIUM

● FIXED

Exact-output swaps are charged on a smaller fee base than exact-input, bypassing the launch-window anti-snipe premium

CashCatHook charges fees in two places. `_beforeSwap` handles swaps where ETH is the specified currency (exact-input buys, exact-output sells) and it takes `specifiedAmount × rate` from the gross ETH. `_afterSwap` handles the other side (exact-output buys, exact-input sells) - it takes `ethMoved × rate` from the net ETH the pool actually settled.

For the same trade result, the fee bases differ. An exact-input buy pays $\text{fee} = \text{gross} \times r$, where `gross` includes the fee itself. An exact-output buy pays $\text{fee} = \text{net} \times r$, where `net` is just the pool settlement. The exact-output fee is always smaller by a factor of $(1 - r)$.

At `baseFeeRate = 10,000` (1%), the difference is negligible (~0.01%). At `launchFeeRate = 900,000` (90%), exact-output buys pay 81% less in fees than exact-input for the same tokens received. This directly undermines the anti-snipe premium and a sniper routing through exact-output during the launch window captures the launch discount the high fee was designed to prevent.

The hook is immutable. Lost revenue from launch windows is permanent and uncollectable.

IMPACT

Snipers can bypass the launch-window anti-snipe premium by routing exact-output buys. The effective fee at 90% launch rate drops from $r / (1 - r) = 900\%$ of the net trade to just $r = 90\%$ - a 10× reduction. Every token launched with a high decay fee is vulnerable from block one and the hook cannot be patched.

RECOMMENDATION

Charge exact-output (afterSwap) fees on the gross amount rather than the net, matching the effective rate of exact-input swaps:

```
// in _afterSwap, for ETH-unspecified swaps:
uint256 fee = (ethMoved * currentFeeRate(poolId, sender)) / (FEE_DENOMINATOR -
currentFeeRate(poolId, sender));
```

This makes $\text{fee} / (\text{ethMoved} + \text{fee}) = r / \text{FEE_DENOMINATOR}$, matching the exact-input effective rate.

● LOW SEVERITY · 6 FINDINGS

L-01

LOW

● FIXED

buyTaxRate() and sellTaxRate() understate the actual fee during the launch decay window

CashCatToken stores `taxBps` at initialization, derived from `baseFeeRate` only - the steady-state fee. The hook `currentFeeRate` actually charges up to `launchFeeRate` during the decay window, linearly decaying down to `baseFeeRate` over `launchFeeDecay` seconds.

Since `launchFeeRate` can be as high as 900,000 pips (90%) and `baseFeeRate` can be as low as 10,000 pips (1%), the token `buyTaxRate()` and `sellTaxRate()` views can understate the real fee by up to 90×.

The NatSpec on these functions says "fixed at launch," which is inaccurate when `launchFeeDecay` $\neq$ 0 - the rate starts high and decays.

IMPACT

Tax scanner terminals and trading bots that read `buyTaxRate()` / `sellTaxRate()` from the token contract get a stale, understated fee. Traders relying on these views during the launch window face unexpectedly large slippage. The `_tokenConfig` helper in `CashCatFactory` also truncates pips to bps via integer division (`baseFeeRate / 100`), losing up to 0.99 bps on top.

RECOMMENDATION

Store both rates and the decay duration on the token so scanners can compute the live rate or point them at the hook:

```
function buyTaxRate() external view returns (uint256) {
    if (poolId == bytes32(0)) return taxBps;
    return CashCatHook(address(IPoolManager(IHooks(factory).poolManager())).getHook(poolId))
        .currentFeeRate(PoolId.wrap(poolId), msg.sender) / 100;
}
```

maybe simpler fix is to store `launchFeeRate`, `launchFeeDecay` and `launchTime` on the token and let the view compute the decay itself, matching the hook formula.

L-02

LOW

● FIXED

CashCatSelfBurner.register does not assert key.toId() == poolId

CashCatSelfBurner.register stores `_keys[poolId] = key` without verifying that the supplied `PoolKey` actually hashes to the supplied `poolId`. If they don't match, `burn` claims fees from one pool (via `hook.claim(poolId)`) and swaps on a completely different pool (the one defined by `key`). This cross-pool confusion is silent and nothing reverts.

The current factory always passes a matching pair because it derives `poolId = key.toId()` before calling `register`. But the factory is a UUPS proxy and a future upgrade that constructs the arguments differently would silently activate this path.

The hook `register` and the distributor `register` don't have this issue because they don't store a separate `PoolKey`.

IMPACT

If triggered by a future factory upgrade, fees from pool A would fund buy-and-burns on pool B token. Pool A token never gets burned; pool B token gets burned with someone else fees. The mismatch is permanent since `register` is one-shot (`AlreadyRegistered`).

RECOMMENDATION

Add the assertion in `register`:

```
function register(PoolId poolId, PoolKey calldata key) external {
    if (msg.sender != factory) revert NotFactory();
    if (key.toId() != poolId) revert PoolKeyMismatch();
    if (Currency.unwrap(_keys[poolId].currency1) != address(0)) revert AlreadyRegistered();
    _keys[poolId] = key;
}
```

L-03

LOW

● FIXED

CashCatSelfBurner.burn is missing nonReentrant

`burn` pays the caller a bounty with a low level `call` that hands control to `msg.sender`, but the function has no reentrancy guard. Every other bounty and payout path in the system is guarded: the sibling `CashCatBuybackBurner.burn`, `CashCatHook.claim`, and the distributor payouts all use `nonReentrant`. `CashCatSelfBurner` does not even inherit `ReentrancyGuard`.

[https://github.com/SB-Security/audit-2026-07-](https://github.com/SB-Security/audit-2026-07-CashCat/blob/98a923708dca83b06a1f90a25d8351c9015d4f95/src/v4/CashCatSelfBurner.sol#L86-L125)

[CashCat/blob/98a923708dca83b06a1f90a25d8351c9015d4f95/src/v4/CashCatSelfBurner.sol#L86-L125](https://github.com/SB-Security/audit-2026-07-CashCat/blob/98a923708dca83b06a1f90a25d8351c9015d4f95/src/v4/CashCatSelfBurner.sol#L86-L125)

```
function burn(PoolId poolId) external returns (uint256 tokensBurned) {
    ...
    if (bounty > 0) {
        (bool paid,) = msg.sender.call{value: bounty}("");
        if (!paid) revert BountyPayFailed();
    }
    emit Burned(poolId, Currency.unwrap(key.currency1), ethSpent, tokensBurned, bounty);
}
```

The `_lastBurnBlock[poolId] == block.number` gate blocks reentry into the same pool, and state is updated before the external call, so this is not directly exploitable today. The guard is missing purely by omission relative to the rest of the codebase, which leaves the contract one refactor away from a real reentrancy path if the block gate or the effects ordering ever change.

IMPACT

No direct exploit today. The risk is defense in depth: `burn` is the only bounty payer in the system without a reentrancy guard, so it is inconsistent with the established pattern and fragile against future changes.

RECOMMENDATION

Inherit `ReentrancyGuard` and add `nonReentrant` to `burn`, matching `CashCatBuybackBurner`.

```
import {ReentrancyGuard} from "@openzeppelin/contracts/utils/ReentrancyGuard.sol";

contract CashCatSelfBurner is IUnlockCallback, ReentrancyGuard {
    ...
    function burn(PoolId poolId) external nonReentrant returns (uint256 tokensBurned) {
```

L-04

LOW

● FIXED

Bounty in `burn` is calculated on requested input instead of ETH actually spent

`CashCatSelfBurner.burn` derives the keeper bounty from `ethIn`, the amount pulled from the fuel tank before the swap. Its sibling `CashCatBuybackBurner.burn` derives the bounty from `ethSpent`, the amount the swap actually consumed.

`CashCatSelfBurner.sol#L103-L104`

```
uint256 bounty = (ethIn * BOUNTY_BPS) / 10_000;  
uint256 burnEth = ethIn - bounty;
```

`CashCatBuybackBurner.sol#L206-L207`

```
uint256 bounty = (ethSpent * BOUNTY_BPS) / BPS_DENOMINATOR;  
burnTank += chunk - ethSpent - bounty;
```

IMPACT

The `bounty` is carved out of `ethIn`, which was already deducted from `unburned`, so ETH accounting stays balanced. Today the burn swap is ETH specified, so the hook rejects any partial fill with `PartialFillRejected` and `ethSpent` always equals `burnEth`, making the refund branch dead. On this only reachable path the bounty is 1% of `ethIn` versus the sibling's 1% of `ethSpent`, an extra $0.0001 * \text{ethIn}$, bounded by `MAX_BURN_PER_CALL` to at most about $1e-5$ ETH per pool per block, paid from burn fuel.

The concern is robustness: if the hook partial fill guard is ever loosened, the refund branch at line 112 becomes live and a keeper would collect the full 1% of `ethIn` while only a fraction of `burnEth` actually bought and burned tokens, letting the tank be drained through bounties without matching burns.

RECOMMENDATION

Compute the bounty from `ethSpent` after the swap, matching `CashCatBuybackBurner.burn`, and return the unused reserve to the tank.

```
uint256 bounty = (ethSpent * BOUNTY_BPS) / 10_000;  
unburned[poolId] += (ethIn - bounty) - ethSpent;
```

L-05

LOW

● FIXED

ALLOW_DEPLOYER_ADMIN is read from ambient .env state and disables the only check that OWNER is not the deployer

DeployV4 guards the production deploy with an identity check on the admin address and an escape hatch that removes it.

DeployV4.s.sol

```
if (block.chainid == 4663 && !vm.envOr("ALLOW_DEPLOYER_ADMIN", false)) {
  owner = vm.envAddress("OWNER");
  teamWallet = vm.envAddress("TEAM_WALLET");
  require(owner != deployer, "DeployV4: OWNER must be the Safe, not the deployer");
  require(owner != address(0) && teamWallet != address(0), "DeployV4: OWNER/TEAM_WALLET
required");
} else {
  owner = vm.envOr("OWNER", deployer);
  teamWallet = vm.envOr("TEAM_WALLET", owner);
  if (owner == deployer) console.log("WARNING: OWNER not set - deployer EOA keeps admin");
  if (teamWallet == owner) console.log("WARNING: TEAM_WALLET not set - team share pays to
OWNER");
}
```

Three facts compose.

The script own comment states that rehearsal forks keep the production chain id, so the hatch is the intended way to rehearse and naturally lives in a developer .env.

DeployV4.s.sol

```
// Robinhood Chain only - a bare --broadcast against the wrong RPC must
// fail, not mint a stray production stack. Forks of the chain keep its
// id, so the local anvil fork passes; override consciously if ever
// rehearsing elsewhere.
```

forge script loads .env from the project root automatically, with no flag and no output saying it did. Verified with a minimal scratch project containing only ALLOW_DEPLOYER_ADMIN=true in .env and no other configuration:

```
└─ ← [Revert] ESCAPE HATCH PICKED UP FROM .env
```

.gitignore lists .env, so it is untracked local state that persists invisibly across sessions.

The hatch branch drops the only sanity check on OWNER and nothing anywhere checks owner.code.length > 0.

IMPACT

A production --broadcast with a stale rehearsal .env transfers ownership of the factory, the hook and the buyback burner to whatever OWNER holds, while printing **no warning** - both console.log conditions are false when OWNER and TEAM_WALLET are set to anything - and the same !! OWNERSHIP IS PENDING banner a correct run prints. If OWNER is a rehearsal value such as an anvil account, its private key is published in Foundry documentation and whoever holds it can acceptOwnership() and then reach factory.upgradeToAndCall, hook.setTreasury and buyback.migrate.

The degenerate variant is milder but quieter: with OWNER unset, owner == deployer, so if (owner != deployer) is false, **no ownership transfer happens at all** and the "OWNERSHIP IS PENDING" banner never prints. The operator sees a clean log while the whole stack is permanently deployer-EOA-owned.

L-05 · CONTINUED

Worth stating for scope: even full admin capture cannot reach an already-launched pool. From the factory address, `hook.register` reverts `AlreadyRegistered`, `updateCreator` and `claim` revert `NotCreator` and the creator tab remains claimable in full. The exposure is platform revenue and control of future launches.

Rated Low because it requires a stale environment variable rather than a code defect. The `EXPECTED_CHAIN_ID` guard by contrast fails closed and is correct.

RECOMMENDATION

Keep the identity check on the production chain even when the hatch is set. The hatch may relax "must be set"; it should never relax "must be a Safe".

```
} else {  
  
    owner = vm.envOr("OWNER", deployer);  
  
    teamWallet = vm.envOr("TEAM_WALLET", owner);  
  
+   if (block.chainid == 4663) require(owner.code.length > 0, "DeployV4: OWNER must be a  
contract (Safe)");  
  
    if (owner == deployer) console.log("WARNING: OWNER not set - deployer EOA keeps admin");
```

Consider also moving the hatch out of ambient state, for example to a `--sig "run(bool)"` argument, so it cannot survive in a `.env` between sessions.

L-06

LOW

● FIXED

updateCreator accepts an address that cannot receive ETH, permanently locking the pool fee stream

updateCreator validates only that the new recipient is non-zero.

CashCatHook.sol

```
function updateCreator(PoolId poolId, address newCreator) external {
    PoolConfig storage config = poolConfigs[poolId];
    if (!config.exists) revert UnknownPool();
    if (msg.sender != config.creator) revert NotCreator();
    if (newCreator == address(0)) revert ZeroAddress();
    emit CreatorUpdated(poolId, config.creator, newCreator);
    config.creator = newCreator;
}
```

claim is the only exit from tab and it pays with _payEth, which reverts on failure.

CashCatHook.sol

```
function _payEth(address to, uint256 amount) private {
    if (amount == 0) return;
    (bool ok,) = to.call{value: amount}("");
    if (!ok) revert EthTransferFailed();
}
```

So a creator who hands the stream to a contract that cannot accept plain ETH - a token, an NFT, a vault with no payable fallback - makes claim revert forever. The failure is silent at the moment it is caused: updateCreator succeeds and nothing breaks until the next claim. The old creator cannot undo it, because updateCreator is gated on being the current creator and the new recipient can only fix it if it happens to be able to make arbitrary calls.

The stream does not stop either. sweep stays permissionless and keeps moving pending into a tab nobody can drain, so the amount locked grows with every trade.

The hand-off is actively encouraged.

[guide/fees.md](#)

standard - the creator wallet. They claim whenever they like and updateCreator lets them hand the stream to another address (a multisig, a contract, a co-founder) without asking anyone.

Nothing in that sentence or in the function own NatSpec, notes that the target must be able to receive ETH.

IMPACT

Permanent loss of the pool entire future fee stream, growing over time. Rated Low because it requires the creator to pick an unsuitable address and the affected funds are that creator own earnings rather than user deposits. The dividend and self-burn modes are unaffected, since their recipients are stamped at birth and cannot call updateCreator at all.

RECOMMENDATION

Let the recipient nominate where the ETH lands, so a contract that cannot receive ETH but can call the hook still recovers the stream. Keep the existing one-argument form as a wrapper.

L-06 · CONTINUED

```
function claim(PoolId poolId, address to) public nonReentrant returns (uint256 amount) {
    PoolConfig storage config = poolConfigs[poolId];
    if (!config.exists) revert UnknownPool();
    if (msg.sender != config.creator) revert NotCreator();
    if (to == address(0)) revert ZeroAddress();
    _sweep(poolId);
    amount = tab[poolId];
    tab[poolId] = 0;
    _payEth(to, amount);
    emit CreatorFeesClaimed(poolId, to, amount);
}

function claim(PoolId poolId) external returns (uint256) {
    return claim(poolId, msg.sender);
}
```

A cheaper alternative, if the signature must not change, is to require `newCreator.code.length == 0 || <the caller asserts it can receive>` - but that wrongly rejects multisigs, which are the documented use case. The pull-based form above is the one that does not trade away a legitimate flow.

● INFORMATIONAL SEVERITY · 1 FINDING

I-01

INFORMATIONAL

● FIXED

setLaunchConfigEnabled is missing a config existence check

Every other function that takes a `configId` checks it against `_launchConfigs.length` and reverts with the custom `InvalidConfigId` error when out of range. `_launch`, `predictTokenAddress` and `mineSalt` all do this.

`setLaunchConfigEnabled` does not. It indexes `_launchConfigs` directly with no bound check.

CashCatFactory.sol#L497-L500

```
function setLaunchConfigEnabled(uint256 configId, bool enabled) external onlyOwner {  
    _launchConfigs[configId].enabled = enabled;  
    emit LaunchConfigEnabled(configId, enabled);  
}
```

For comparison, the guard used everywhere else:

CashCatFactory.sol#L398

```
if (configId ≥ _launchConfigs.length) revert InvalidConfigId();
```

IMPACT

No security impact. The function is `onlyOwner`. Passing an out of range `configId` reverts on the array access with a generic `Panic(0x32)` instead of the clear `InvalidConfigId` error, which is inconsistent with the rest of the contract and harder to read off chain.

RECOMMENDATION

Add the same bound check the other functions use:

```
function setLaunchConfigEnabled(uint256 configId, bool enabled) external onlyOwner {  
    if (configId ≥ _launchConfigs.length) revert InvalidConfigId();  
    _launchConfigs[configId].enabled = enabled;  
    emit LaunchConfigEnabled(configId, enabled);  
}
```

● GAS OPTIMISATION SEVERITY · 2 FINDINGS

G-01

GAS OPT.

● FIXED

launchFeeDecay can be non-zero while baseFeeRate == launchFeeRate writing dead state

The launch fee is meant to decay from `launchFeeRate` down to `baseFeeRate`. When those two are equal there is nothing to decay `currentFeeRate` returns `baseFeeRate` at every timestamp, whatever `launchFeeDecay` is.

`register` allows that contradiction: its only fee check is `baseFeeRate > launchFeeRate`, so it accepts `baseFeeRate == launchFeeRate` together with a non-zero `launchFeeDecay`, never tying the two. Equal rates are fine, but a non-zero decay on top of them is the issue, the decay is meaningless, yet because `launchFeeDecay != 0`, `_consumeFirstBuyExemption` still writes `firstBuyExemptionSpent[poolId] = true` on the factory's first buy. That flag is only ever read to return `baseFeeRate`, which everyone already pays.

IMPACT

No fee ever changes. The cost is a misleading `public` flag (`firstBuyExemptionSpent == true` on a pool with no real exemption) plus wasted gas. A ~20k `SSTORE` and decay math on every swap to reproduce what `launchFeeDecay = 0` gives for free.

RECOMMENDATION

Equal rates and a non-zero decay are contradictory; collapse them in `register`

```
// nothing to decay when the rates are equal
if (baseFeeRate == launchFeeRate) launchFeeDecay = 0;
```

G-02

GAS OPT.

● FIXED

tokensOut == 0 revert runs after the swap already did wasted work in the callback

burn decodes the result of the unlock and only then checks whether the swap returned any tokens:

CashCatSelfBurner.sol#L104-L117

```
(uint256 tokensOut, uint256 ethSpent) =
    abi.decode(poolManager.unlock(abi.encode(key, burnEth)), (uint256, uint256));
if (ethSpent < burnEth) {
    unburned[poolId] += burnEth - ethSpent;
}
if (tokensOut == 0) revert NothingToBurn();
```

By the time this check runs, unlockCallback has already settled the ETH, taken the tokens, and called burn with a zero amount:

CashCatSelfBurner.sol#L139-L145

```
uint256 tokensOut = uint256(uint128(delta.amount1()));
uint256 ethSpent = uint256(uint128(-delta.amount0()));
key.currency0.settle(poolManager, address(this), ethSpent, false);
key.currency1.take(poolManager, address(this), tokensOut, false);
CashCatToken(Currency.unwrap(key.currency1)).burn(tokensOut);
return abi.encode(tokensOut, ethSpent);
```

When tokensOut is 0, the take of zero tokens, the burn of zero, the abi.encode, the abi.decode, and the refund branch in burn all execute before the transaction reverts and rolls everything back. That work is wasted.

IMPACT

Only the revert path is affected. A keeper firing a burn on a pool sitting at its price floor pays for the swap, settle, take, burn, and encode/decode before hitting the revert, instead of stopping the moment tokensOut is known to be zero. The happy path is unchanged, so there is no saving during normal operation. This is a minor gas refinement, not a correctness issue.

RECOMMENDATION

Move the zero check into unlockCallback, right after tokensOut is computed, so the swap bails before the settle, take, and burn:

```
uint256 tokensOut = uint256(uint128(delta.amount1()));
if (tokensOut == 0) revert NothingToBurn();
uint256 ethSpent = uint256(uint128(-delta.amount0()));
```

The behavior is identical because poolManager.unlock bubbles the revert up and rolls back the fuel deduction either way. If you keep the check in burn instead, that is also fine given how marginal the saving is.

08 POST-AUDIT STATEMENT

Post-Audit Statement

This engagement surfaced 13 findings of Medium severity or above, of which 12 are withheld from this version of the report (see below). SBSecurity applies a fixed, protocol-independent policy to every audit it delivers, and this section records how that policy applies to the present report.

WITHHELD FINDINGS

Fifteen findings from this engagement (7 High, 5 Medium, 1 Low, 2 Informational; 11 fixed, 4 acknowledged) are withheld from this version at the client's request. They are recorded in full in the complete report delivered to the client on 22.08.2026, whose SHA-256 is 8c9a981fe47bd1c65e38b5c1b2f0b8d04d8ff503b3fe61fb1876350495ce1423. The withheld findings are counted in the figures of this statement but not in the summary and findings sections of this document.

RE-AUDIT REQUIREMENT

As per SBSecurity guidelines, any audit that yields ten or more findings of Medium severity or above requires the codebase to undergo a full re-audit before this report can be relied upon as a statement of its overall security posture. This threshold reflects the well-established relationship between finding density and residual risk: when remediation involves this volume of change, the changes themselves become a source of risk that a fix review alone cannot retire.

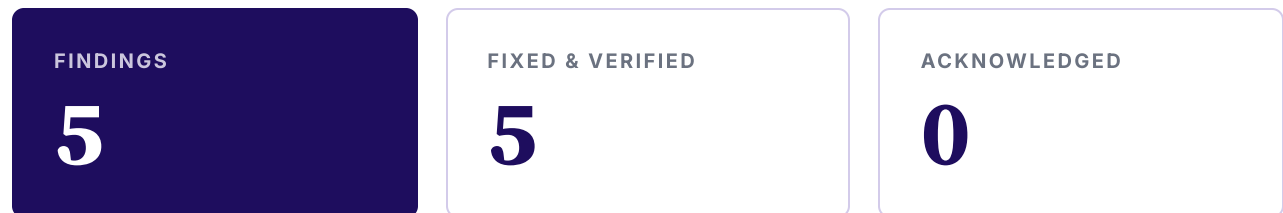
A NOTE ON THIS POLICY

This requirement is procedural and applies uniformly across SBSecurity engagements. It is not a judgment of the letscash codebase or team, whose responsiveness throughout remediation resulted in 21 of the 25 findings of the engagement being resolved within the remediation window. A follow-up engagement is the standard and recommended path to a clean security posture statement.

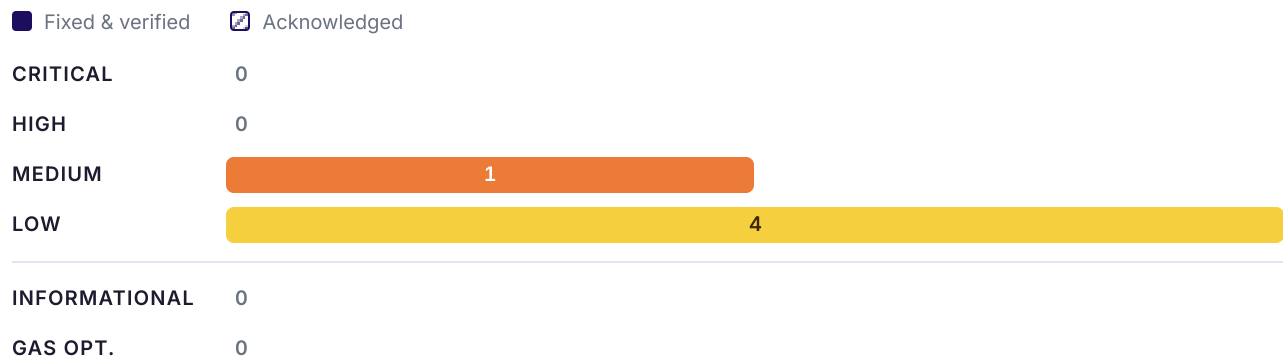
A APPENDIX · VNEXT FOLLOW-UP REVIEW

vNext findings

This appendix records a follow-up review of the vNext contracts, which entered the repository after the preceding report was finalized. It is issued as an addendum: the report body is unchanged and the appendix carries its own page numbering. Finding identifiers continue the numbering of the main report.



SEVERITY DISTRIBUTION



REVIEWED COMMIT	REVIEW WINDOW	METHODS
77d08fa9	05.08.2026 - 06.08.2026	Manual review

CONTRACTS IN SCOPE

CashCatFactoryVNext.sol	CashCatHookV2.sol
CashCatLaunchSplitter.sol	CashCatRevenueConverter.sol
CashCatSelfBurnerV2.sol	CashCatTokenV2.sol
ExactTransfer.sol	

REVIEWED COMMIT HASH 77d08fa93fb6b39432446290c65f8ef47ab4b489

Remediation review

All five findings in this appendix were remediated after the review window. The commits listed below are the only changes this remediation review covers: L-07, L-08 and L-09 were closed by changes to the reviewed contracts, and M-02 and L-10 by replacing the affected contracts, which were reviewed as complete contracts rather than as diffs against their predecessors. Repository changes outside these commits, including everything that landed between the review window and the fix commits, were not reviewed and are not covered by this appendix.

FIX REVIEW WINDOW	FIX COMMITS	METHODS
06.08.2026	14 across 5 findings	Manual review
M-02	54787911a94732c2f533561db5c917e203c93a47	
	18.08.2026 · Add a price floor to the revenue splitter buys	
	9aa90799c1a1b5afe270c764b076fd8f5ca557d6	
	18.08.2026 · Test the splitter fix against the deployed contract	
L-07	b0ecb5fd2af0da0a163cf88c699328a9459a4f65	
	19.08.2026 · Size buys to live depth, and check the oracle at construction	
	00a55a607f14e0afb676aad75e9a2d0319e38da8	
	19.08.2026 · Correct the oracle buffer floor, and name a quiet lane	
L-08	6f60c0674ae617305a9e2379ae7a58cc297b155d	
	05.08.2026 · Refuse a quote whose base unit is too big for the fee to round on	
	7fb5c15478ac6abecb3d424fec719a224243a0e0	
L-08	4269b0e699bf68338a3b6b1d96b684d11e1dff4f	
	05.08.2026 · Argue against both fixes, and tighten the one that lost	
	67d4e25761d512357ee72305579f127774680e1e	
L-08	05.08.2026 · Let a claim and a collect name an amount	

L-09

ca7a758324b404d2b29a78ff48606b235cc2dd27

05.08.2026 · Validate three config values where they are set

eeb12908e0b7e1196f174581ec39620ed3cd262e

05.08.2026 · Refuse every address of ours as a fee recipient, and read the batch harder

bf3423c03b73e31caafb794904f6905021f59ad8

05.08.2026 · Bound the TWAP window by the history the ring can hold

7b30716a84386ebc656fde8d09b5e355b6139131

05.08.2026 · Require the ring to hold the window now, not eventually

L-10

a349f4ed0f956cfc0b5e24227ba9b91dcc0c1d42

19.08.2026 · Size converter sales to live depth, and report a stuck route

8ee4c293e6d3327a12d717185733a8c4eb44d3d6

19.08.2026 · Refuse a fee tier the band cannot cover, and name why a route is quiet

REPLACEMENT CONTRACTS REVIEWED IN FULL

CashCatRevenueSplitterV2.sol (778 lines) replaces the 405-line CashCatRevenueSplitter and closes M-02. Lane buys are priced against a 2-minute average and spot with the pool fee removed and a 100 bps band (BelowFloor), the chunk is capped at 25 bps of the pool's WETH reserve (VenueTooThin), and setMaxBuyPerCall is bounded by the constant MAX_BUY_CEILING of 1 ETH, so the cap is an invariant rather than a storage slot.

CashCatRevenueConverterV2.sol (749 lines) replaces the 501-line CashCatRevenueConverter and closes L-10. convertibleNow is now the minimum of the booked tab, maxPerCall and a live depth reading, so a sale shrinks with liquidity instead of pinning at a figure measured once, and a serviceable view reports why a route is not clearing.

CHANGES OUTSIDE THE REVIEW SCOPE

- The splitter changeover (script/DeployRevenueSplitterV2.s.sol, vnext/SPLITTER-V2-CHANGEOVER.md), which repoints four treasury pointers (vNext hook, retired hook, USDG converter, factory) to the new splitter. This is deployment procedure, not contract logic.
- Validation added beyond the findings: an oracle readiness check at splitter construction (OracleNotReady) and a fee-tier check in the converter's setRoute (BandTooTightForPool).
- Scripts, tests and documentation added alongside both fixes (check-batch.js, VerifyVNextRelease.s.sol, new fork tests, vnext/DEFERRED.md).
- Converter routes remain keyed by the quote token alone (L-09, item 8). The re-keying is scheduled for the batch that redeploys the converter and is not part of CashCatRevenueConverterV2.
- Unmerged feature branches (feature/airdrop-vault, feature/supply-100b-1t) and the ten-billion-supply configuration work on main are outside this appendix.

What we found

ID	TITLE	SEVERITY	STATUS
M-02	<code>CashCatRevenueSplitter</code> buys CASHCAT with no minimum output, no oracle floor and no depth-relative bound, while paying the caller to trigger it	MEDIUM	● FIXED
L-07	Swap fee rounds down to zero for small quote legs, which becomes meaningful if a low decimal quote is ever approved	LOW	● FIXED
L-08	A dust credit from any address freezes a launch splitter's all-or-nothing claim, if a quote that caps transfer size is ever approved	LOW	● FIXED
L-09	Configuration values are accepted at the point they are set but not honoured at the point they are used	LOW	● FIXED
L-10	The converter static per-call ceiling permanently strands a route platform fees, if a thinly-traded quote is ever approved	LOW	● FIXED

● MEDIUM SEVERITY · 1 FINDING

M-02

MEDIUM

● FIXED

CashCatRevenueSplitter buys CASHCAT with no minimum output, no oracle floor and no depth-relative bound, while paying the caller to trigger it

Both revenue lanes are permissionless and pay the caller a share of what they spend:

[https://github.com/SB-Security/audit-2026-07-](https://github.com/SB-Security/audit-2026-07-CashCat/blob/77d08fa93fb6b39432446290c65f8ef47ab4b489/src/v4/CashCatRevenueSplitter.sol#L220-L230)

[CashCat/blob/77d08fa93fb6b39432446290c65f8ef47ab4b489/src/v4/CashCatRevenueSplitter.sol#L220-L230](https://github.com/SB-Security/audit-2026-07-CashCat/blob/77d08fa93fb6b39432446290c65f8ef47ab4b489/src/v4/CashCatRevenueSplitter.sol#L220-L230)

```
/// @notice Buys CASHCAT with the burn tank and sends it to the dead
///         address. Callable by anyone; pays the caller the bounty.
function buyAndBurn() external nonReentrant returns (uint256 cashcatOut) {
    return _buy(BURN_ADDRESS, true);
}

/// @notice Buys CASHCAT with its lane's tank and sends it to the CashCat
///         treasury. Callable by anyone; pays the caller the bounty.
function buyForCashcatTreasury() external nonReentrant returns (uint256 cashcatOut) {
    return _buy(cashcatTreasury, false);
}
```

Neither takes a minimum output. Inside, the swap is sent with the extreme sqrt price bound, which is not a price limit but the absence of one, and the only thing asserted about the result is that it is not zero:

[https://github.com/SB-Security/audit-2026-07-](https://github.com/SB-Security/audit-2026-07-CashCat/blob/77d08fa93fb6b39432446290c65f8ef47ab4b489/src/v4/CashCatRevenueSplitter.sol#L255-L268)

[CashCat/blob/77d08fa93fb6b39432446290c65f8ef47ab4b489/src/v4/CashCatRevenueSplitter.sol#L255-L268](https://github.com/SB-Security/audit-2026-07-CashCat/blob/77d08fa93fb6b39432446290c65f8ef47ab4b489/src/v4/CashCatRevenueSplitter.sol#L255-L268)

```
bool zeroForOne = _wethIsToken0; // WETH in
(int256 amount0, int256 amount1) = pool.swap(
    destination,
    zeroForOne,
    int256(buyEth),
    zeroForOne ? MIN_SQRT_RATIO_PLUS_1 : MAX_SQRT_RATIO_MINUS_1,
    "");
);
uint256 ethSpent = uint256(zeroForOne ? amount0 : amount1);
cashcatOut = uint256(-(zeroForOne ? amount1 : amount0));
// No bounty for buying nothing: reverting rolls the tank deduction back
// too, so an empty pool cannot be milked for bounties.
if (cashcatOut == 0) revert NothingToBuy();
```

So the contract will accept one wei of CASHCAT for a full chunk of ETH. There is no oracle reference, no deviation bound, and no check on the depth being traded against.

The sibling contract in this same release does all three for its own sale. `CashCatRevenueConverter.convert` derives a floor from the maximum of a long TWAP, a short TWAP and spot, discounts it by a bounded slippage, and additionally lets the caller tighten it:

[https://github.com/SB-Security/audit-2026-07-](https://github.com/SB-Security/audit-2026-07-CashCat/blob/77d08fa93fb6b39432446290c65f8ef47ab4b489/src/v4/CashCatRevenueConverter.sol#L347-L351)

[CashCat/blob/77d08fa93fb6b39432446290c65f8ef47ab4b489/src/v4/CashCatRevenueConverter.sol#L347-L351](https://github.com/SB-Security/audit-2026-07-CashCat/blob/77d08fa93fb6b39432446290c65f8ef47ab4b489/src/v4/CashCatRevenueConverter.sol#L347-L351)

M-02 · CONTINUED

```
function _floorFor(Route memory r, uint256 amountIn) private view returns (uint256) {
    (int24 longTick, int24 shortTick) = _averageTicks(r);
    uint256 fair = _fairValue(r, longTick, shortTick, amountIn);
    return (fair * (BPS_DENOMINATOR - r.maxSlippageBps)) / BPS_DENOMINATOR;
}
```

The splitter buys from a Uniswap v3 pool, the same venue class the converter sells into, so the oracle the converter reads is available to it on identical terms.

Three details determine how much this is worth to an attacker, and each cuts against the size cap being treated as the mitigation.

The trigger is the attacker. Both entry points are permissionless and pay `BOUNTY_BPS`, one percent of the chunk. The attacker does not need to observe a pending transaction or win an ordering race, because they submit the buy themselves, in one transaction, between their own two trades. They are also paid for doing it.

The per-block gate does not bind what one event extracts. The two lanes hold independent gates, so two chunks are available per block rather than one, and forcing accrual between calls guarantees each permitted buy is a full cap-sized chunk, which is the most profitable size to trade against.

The cap is a setting, not an invariant. `maxBuyPerCall` is mutable storage with an owner setter, unlike the self-burner's `MAX_BURN_PER_CALL`, which is `constant`:

[https://github.com/SB-Security/audit-2026-07-](https://github.com/SB-Security/audit-2026-07-CashCat/blob/77d08fa93fb6b39432446290c65f8ef47ab4b489/src/v4/CashCatRevenueSplitter.sol#L128)

[CashCat/blob/77d08fa93fb6b39432446290c65f8ef47ab4b489/src/v4/CashCatRevenueSplitter.sol#L128](https://github.com/SB-Security/audit-2026-07-CashCat/blob/77d08fa93fb6b39432446290c65f8ef47ab4b489/src/v4/CashCatRevenueSplitter.sol#L128)

```
uint256 public maxBuyPerCall;
```

IMPACT

Protocol revenue is converted at a price an unprivileged caller chooses, and the difference is theirs. The loss lands on whichever lane was spent, so either fewer tokens are burned than the burn budget paid for, or the letscash treasury receives less than it was owed.

What an absolute cap cannot do is bound this, because extraction scales with the buy's size relative to the depth it trades against, not with its size in ether. Modelling a 0.495 ETH chunk, the cap of 0.5 ETH less the bounty reservation, against a full-range CASHCAT pool at the 0.3 percent tier with the front-run size swept to its optimum:

pool depth	extracted per buy	share of the buy
5 ETH	0.379 ETH	76%
25 ETH	0.221 ETH	45%
50 ETH	0.124 ETH	25%
100 ETH	0.031 ETH	6.2%
250 ETH	about 0	about 0

Extraction stays above five percent of every buy until the pool holds roughly 106 ETH at that tier, or roughly 31 ETH at the one percent tier. Below those depths this is not a rounding loss, it is most of the buy.

Whether that is happening today is therefore a fact about the CASHCAT pool rather than about this contract, and the contract contains nothing that would notice if the answer changed. Liquidity providers withdrawing is ordinary behaviour and needs no bad intent, so a deployment that is safe at listing can become unsafe without any transaction touching the protocol.

RECOMMENDATION

Give the buy a floor derived from something the caller cannot move inside their own transaction. The converter's `_floorFor` is the pattern to reuse, and reusing it keeps the two revenue paths consistent rather than introducing a second convention.

M-02 · CONTINUED

A caller-supplied `minOut` alone is not sufficient here, and is worth stating explicitly because it is the obvious first fix. The caller in the case that matters is the attacker, and they will pass zero.

Two additions are worth making alongside it. Bound the chunk against live depth rather than only against an absolute constant, so that the protection does not silently weaken as the pool thins. And consider making bounty eligibility conditional on the execution clearing the oracle-derived floor, so that the incentive to run the crank is aligned with running it at a fair price rather than merely with running it.

● LOW SEVERITY · 4 FINDINGS

L-07

LOW

● FIXED

Swap fee rounds down to zero for small quote legs, which becomes meaningful if a low decimal quote is ever approved

The hook computes the swap fee with truncating division:

[https://github.com/SB-Security/audit-2026-07-](https://github.com/SB-Security/audit-2026-07-CashCat/blob/77d08fa93fb6b39432446290c65f8ef47ab4b489/src/v4/CashCatHookV2.sol#L255-L259)

[CashCat/blob/77d08fa93fb6b39432446290c65f8ef47ab4b489/src/v4/CashCatHookV2.sol#L255-L259](https://github.com/SB-Security/audit-2026-07-CashCat/blob/77d08fa93fb6b39432446290c65f8ef47ab4b489/src/v4/CashCatHookV2.sol#L255-L259)

```
function _feeFor(uint256 ethAmount, uint256 rate, bool exactOutput) private pure returns (uint256) {
    return exactOutput
        ? (ethAmount * rate) / (FEE_DENOMINATOR - rate)
        : (ethAmount * rate) / FEE_DENOMINATOR;
}
```

The fee is zero whenever the quote leg is below `FEE_DENOMINATOR / feeRate` base units. At a typical 1% rate (`feeRate = 10_000`) that is any leg under 100 base units, and at the 1 pip floor allowed by `register` it is any leg under `1e6` base units.

Whether that window is worth anything depends entirely on the value of one base unit of the quote asset, which is a function of its decimals. `setApprovedQuote` checks that the quote is a contract but nothing about its decimals:

[https://github.com/SB-Security/audit-2026-07-](https://github.com/SB-Security/audit-2026-07-CashCat/blob/77d08fa93fb6b39432446290c65f8ef47ab4b489/src/v4/CashCatFactoryVNext.sol#L1121-L1126)

[CashCat/blob/77d08fa93fb6b39432446290c65f8ef47ab4b489/src/v4/CashCatFactoryVNext.sol#L1121-L1126](https://github.com/SB-Security/audit-2026-07-CashCat/blob/77d08fa93fb6b39432446290c65f8ef47ab4b489/src/v4/CashCatFactoryVNext.sol#L1121-L1126)

```
function setApprovedQuote(address quote, bool approved) external onlyOwner {
    if (quote == address(0)) revert InvalidConfig();
    if (approved && quote.code.length == 0) revert NotAContract();
    approvedQuote[quote] = approved;
    emit QuoteApprovalUpdated(quote, approved);
}
```

For every plausible listing today the window is dust: with 18 decimal ETH or a 6 decimal stable, 100 base units is a fraction of a cent, and gas makes grinding it pointless. But for a low decimal quote the same 100 base units is real money. With a 2 decimal stable like GUSD it is \$1 per swap; with a 0 decimal token it is 100 whole tokens. A router that splits a trade into legs just under the threshold pays no fee at all.

IMPACT

No impact with the assets likely to be approved. If a quote with very few decimals is ever listed, the fee stream of every pool launched against it can be shaved systematically by splitting trades, and the creator and platform shares both bleed. The listing path has no guard that would surface this at approval time.

RECOMMENDATION

Enforce a minimum decimals check in `setApprovedQuote` so a low decimal quote cannot be listed:

L-07 · CONTINUED

```
uint8 public constant MIN_QUOTE_DECIMALS = 6;

function setApprovedQuote(address quote, bool approved) external onlyOwner {
    if (quote == address(0)) revert InvalidConfig();
    if (approved) {
        if (quote.code.length == 0) revert NotAContract();
        if (IERC20Metadata(quote).decimals() < MIN_QUOTE_DECIMALS) revert QuoteDecimalsTooLow();
    }
    approvedQuote[quote] = approved;
    emit QuoteApprovalUpdated(quote, approved);
}
```

`decimals` is optional in ERC-20, so a token that does not implement it will revert here and cannot be listed. For a mandatory gate that is the right failure mode: a quote whose decimals cannot even be read has no business being the denomination of a fee stream.

L-08

LOW

● FIXED

A dust credit from any address freezes a launch splitter's all-or-nothing claim, if a quote that caps transfer size is ever approved

CashCatLaunchSplitter pays a recipient their whole balance or nothing at all. collect reads the slot's entire owed, zeroes it, and transfers that one figure. There is no amount parameter and no partial overload anywhere on the contract:

[https://github.com/SB-Security/audit-2026-07-](https://github.com/SB-Security/audit-2026-07-CashCat/blob/77d08fa93fb6b39432446290c65f8ef47ab4b489/src/v4/CashCatLaunchSplitter.sol#L238-L245)

[CashCat/blob/77d08fa93fb6b39432446290c65f8ef47ab4b489/src/v4/CashCatLaunchSplitter.sol#L238-L245](https://github.com/SB-Security/audit-2026-07-CashCat/blob/77d08fa93fb6b39432446290c65f8ef47ab4b489/src/v4/CashCatLaunchSplitter.sol#L238-L245)

```
amount = owed[msg.sender];
if (amount == 0) revert NothingOwed();

owed[msg.sender] = 0;
totalCollected += amount;

_pay(to, amount);
emit Collected(msg.sender, to, amount);
```

The figure that comes out of is permissionless to grow. distribute is callable by anyone, and it divides the contract's whole unbooked balance rather than the amount the hook just handed over:

[https://github.com/SB-Security/audit-2026-07-](https://github.com/SB-Security/audit-2026-07-CashCat/blob/77d08fa93fb6b39432446290c65f8ef47ab4b489/src/v4/CashCatLaunchSplitter.sol#L199-L202)

[CashCat/blob/77d08fa93fb6b39432446290c65f8ef47ab4b489/src/v4/CashCatLaunchSplitter.sol#L199-L202](https://github.com/SB-Security/audit-2026-07-CashCat/blob/77d08fa93fb6b39432446290c65f8ef47ab4b489/src/v4/CashCatLaunchSplitter.sol#L199-L202)

```
uint256 booked = totalAllocated - totalCollected;
uint256 held = _balance();
uint256 pot = held > booked ? held - booked : 0;
if (pot == 0) return 0;
```

That is deliberate and the natspec above it says so: anything sent here directly is picked up on the next pass instead of being stranded. The consequence is that a third party can raise another party's owed simply by transferring the quote asset to the clone and calling distribute. The pot is divided on the fixed shares, so one donation lifts every slot at once:

[https://github.com/SB-Security/audit-2026-07-](https://github.com/SB-Security/audit-2026-07-CashCat/blob/77d08fa93fb6b39432446290c65f8ef47ab4b489/src/v4/CashCatLaunchSplitter.sol#L210-L218)

[CashCat/blob/77d08fa93fb6b39432446290c65f8ef47ab4b489/src/v4/CashCatLaunchSplitter.sol#L210-L218](https://github.com/SB-Security/audit-2026-07-CashCat/blob/77d08fa93fb6b39432446290c65f8ef47ab4b489/src/v4/CashCatLaunchSplitter.sol#L210-L218)

```
for (uint256 i; i < n - 1; ++i) {
    uint256 cut = (pot * shares[i]) / BPS_DENOMINATOR;
    owed[recipients[i]] += cut;
    handedOut += cut;
}
uint256 last = pot - handedOut;
owed[recipients[n - 1]] += last;

totalAllocated += pot;
```

Moving slot `i` by one base unit costs `ceil(BPS_DENOMINATOR / shares[i])` base units. For the last slot it costs exactly one, because that slot takes the remainder rather than its truncated share.

None of this matters while every claim is payable. It starts to matter when the quote asset refuses a transfer above a fixed size. setApprovedQuote gates a listing on two things, and neither is transfer size:

[https://github.com/SB-Security/audit-2026-07-](https://github.com/SB-Security/audit-2026-07-CashCat/blob/77d08fa93fb6b39432446290c65f8ef47ab4b489/src/v4/CashCatFactoryVNext.sol#L1131-L1136)

[CashCat/blob/77d08fa93fb6b39432446290c65f8ef47ab4b489/src/v4/CashCatFactoryVNext.sol#L1131-L1136](https://github.com/SB-Security/audit-2026-07-CashCat/blob/77d08fa93fb6b39432446290c65f8ef47ab4b489/src/v4/CashCatFactoryVNext.sol#L1131-L1136)

L-08 · CONTINUED

```
function setApprovedQuote(address quote, bool approved) external onlyOwner {
    if (quote == address(0)) revert InvalidConfig();
    if (approved && quote.code.length == 0) revert NotAContract();
    approvedQuote[quote] = approved;
    emit QuoteApprovalUpdated(quote, approved);
}
```

Take a quote whose immutable per transfer maximum is `M`, and a recipient holding a fully backed claim of exactly `M`. One base unit credited by anybody takes that claim to `M + 1`. Every `collect` now attempts a transfer above `M` and reverts inside the token. Because the revert unwinds the whole call, the mapping write is rolled back and the same oversized figure is still there. Further accrual and further credits only raise it.

The escapes this contract does offer do not reach the failure. `collect` lets the claimant name a destination, which routes around a recipient blacklist, but does nothing against a size limit, because the revert is keyed to the amount and not to the identity. `rotate` moves the slot to a fresh address, but carries the pending balance across whole rather than dividing it:

[https://github.com/SB-Security/audit-2026-07-](https://github.com/SB-Security/audit-2026-07-CashCat/blob/77d08fa93fb6b39432446290c65f8ef47ab4b489/src/v4/CashCatLaunchSplitter.sol#L270-L272)

[CashCat/blob/77d08fa93fb6b39432446290c65f8ef47ab4b489/src/v4/CashCatLaunchSplitter.sol#L270-L272](https://github.com/SB-Security/audit-2026-07-CashCat/blob/77d08fa93fb6b39432446290c65f8ef47ab4b489/src/v4/CashCatLaunchSplitter.sol#L270-L272)

```
uint256 pending = owed[msg.sender];
owed[msg.sender] = 0;
owed[to] += pending;
```

There is no partial claim, no owner and no rescue, and the contract states the last of those as a property rather than an omission.

IMPACT

A fully backed creator or charity claim becomes permanently unclaimable, for one base unit plus gas. The splitter still holds the money and the ledger still agrees with the balance, so nothing is stolen, and equally nothing is recoverable: the clone cannot be upgraded, has no owner, and exposes no path that moves `owed` anywhere except through the transfer that now reverts.

There is no impact on the assets likely to be approved. ETH has no transfer ceiling, and the stablecoins the suite models exercise freeze, pause and blacklist rather than a size cap. The precondition is a listing decision.

One half of it is not a grief at all. Once such a quote is listed, ordinary fee accrual crosses `M` by itself as soon as a launch earns more than one maximum transfer between collects, and the claim dies with nobody attacking it. The permissionless credit only makes it cheap, immediate and deliberately targeted. So gating the credit path alone would not be a complete fix.

RECOMMENDATION

Add an amount bounded claim. The pattern already exists in this codebase, on the hook's platform lane:

[https://github.com/SB-Security/audit-2026-07-](https://github.com/SB-Security/audit-2026-07-CashCat/blob/77d08fa93fb6b39432446290c65f8ef47ab4b489/src/v4/CashCatHookV2.sol#L618-L620)

[CashCat/blob/77d08fa93fb6b39432446290c65f8ef47ab4b489/src/v4/CashCatHookV2.sol#L618-L620](https://github.com/SB-Security/audit-2026-07-CashCat/blob/77d08fa93fb6b39432446290c65f8ef47ab4b489/src/v4/CashCatHookV2.sol#L618-L620)

```
uint256 booked = platformTab[currency];
if (amount == 0 || amount > booked) revert InsufficientPlatformBalance();
platformTab[currency] = booked - amount;
```

The same shape on the splitter, with the existing `collect` delegating to it for the full balance:

L-08 · CONTINUED

```
function collect(address to, uint256 amount) public nonReentrant returns (uint256) {
    if (to == address(0)) revert ZeroAddress();
    if (to == address(this)) revert SelfReference();
    uint256 booked = owed[msg.sender];
    if (amount == 0 || amount > booked) revert NothingOwed();

    owed[msg.sender] = booked - amount;
    totalCollected += amount;

    _pay(to, amount);
    emit Collected(msg.sender, to, amount);
    return amount;
}
```

Each chunk still pays through `payExact`, so the exactness policy is unchanged and the conservation property still holds. A recipient facing a capped asset can then withdraw in pieces no larger than the cap, which removes the permanent freeze without needing to predict the cap at listing time. The same overload on `CashCatHookV2.claim` closes the creator lane, which is all-or-nothing in the identical way.

Independently, the asset property register above `setApprovedQuote` is the best statement of the asset model in the repository, and it covers exact debit, short delivery, zero delivery, excess debit and address freezing without mentioning transfer size. Adding a clause that an approved quote must not cap the size of a single transfer turns the precondition into an explicit listing rule rather than an unstated assumption. That is worth doing whether or not the partial claim lands, since the two fixes address different halves: the clause prevents the asset being listed, the overload survives it being listed anyway.

L-09

LOW

● FIXED

Configuration values are accepted at the point they are set but not honoured at the point they are used

Nine places accept a configuration value that another part of the system, or the documentation, will not honour. None is reachable by an unprivileged caller: each needs the owner, or in item 9 the launcher, to choose the value. They are grouped because the remedy is identical throughout, which is to validate where the value is set rather than discover the mismatch at first use.

Items 1 and 2 are the ones that fail late rather than at first use, and are the reason this is worth addressing as a group.

1. A TWAP window is accepted that the observation ring cannot hold.

`setRoute` accepts a window up to 86,400 seconds, then requests cardinality through `_cardinalityFor`, which silently clamps to 65,535 slots. On a one second chain a 24 hour window needs about 108,000 slots, so the ring wraps and every conversion on the route reverts once the requested history no longer exists. The install time call to `_averageTicks` confirms history reaches back today, not that the ring can sustain the window.

[https://github.com/SB-Security/audit-2026-07-](https://github.com/SB-Security/audit-2026-07-CashCat/blob/77d08fa93fb6b39432446290c65f8ef47ab4b489/src/v4/CashCatRevenueConverter.sol#L462-L467)

[CashCat/blob/77d08fa93fb6b39432446290c65f8ef47ab4b489/src/v4/CashCatRevenueConverter.sol#L462-L467](https://github.com/SB-Security/audit-2026-07-CashCat/blob/77d08fa93fb6b39432446290c65f8ef47ab4b489/src/v4/CashCatRevenueConverter.sol#L462-L467)

2. `publishConfig` accepts supply and range combinations where floor rounding strands supply.

Seed liquidity is floored, only the amount that rounded liquidity requires is settled, and nothing compares the settled amount against the minted supply, so every launch leaves a residual the owner can later rescue. The residual is bounded by the token amount for one unit of liquidity, which is 27,160 wei at the live tick spacing and start tick but 18445128477729074926 wei at tick spacing 1 near the range extreme, where a supply close to twice that figure strands 49.99% of itself.

[https://github.com/SB-Security/audit-2026-07-](https://github.com/SB-Security/audit-2026-07-CashCat/blob/77d08fa93fb6b39432446290c65f8ef47ab4b489/src/v4/CashCatFactoryVNext.sol#L1193-L1197)

[CashCat/blob/77d08fa93fb6b39432446290c65f8ef47ab4b489/src/v4/CashCatFactoryVNext.sol#L1193-L1197](https://github.com/SB-Security/audit-2026-07-CashCat/blob/77d08fa93fb6b39432446290c65f8ef47ab4b489/src/v4/CashCatFactoryVNext.sol#L1193-L1197)

3. Seed liquidity is bounded by `int128` but not by v4's per tick cap.

`publishConfig` checks only that seed liquidity fits a signed 128 bit value, while v4 separately caps gross liquidity at each initialized tick according to tick spacing. A config can therefore publish and then revert on every launch made against it.

[https://github.com/SB-Security/audit-2026-07-](https://github.com/SB-Security/audit-2026-07-CashCat/blob/77d08fa93fb6b39432446290c65f8ef47ab4b489/src/v4/CashCatFactoryVNext.sol#L1064)

[CashCat/blob/77d08fa93fb6b39432446290c65f8ef47ab4b489/src/v4/CashCatFactoryVNext.sol#L1064](https://github.com/SB-Security/audit-2026-07-CashCat/blob/77d08fa93fb6b39432446290c65f8ef47ab4b489/src/v4/CashCatFactoryVNext.sol#L1064)

4. Fee rate is not restricted to the documented ladder.

The fee ladder is documented as 1%, 3%, 5% and 10%, but validation accepts any nonzero value up to `MAX_FEE_RATE`, so 20,000 pips publishes as a permanent 2% tax that no document describes.

[https://github.com/SB-Security/audit-2026-07-](https://github.com/SB-Security/audit-2026-07-CashCat/blob/77d08fa93fb6b39432446290c65f8ef47ab4b489/src/v4/CashCatFactoryVNext.sol#L1002)

[CashCat/blob/77d08fa93fb6b39432446290c65f8ef47ab4b489/src/v4/CashCatFactoryVNext.sol#L1002](https://github.com/SB-Security/audit-2026-07-CashCat/blob/77d08fa93fb6b39432446290c65f8ef47ab4b489/src/v4/CashCatFactoryVNext.sol#L1002)

5. The factory's treasury setter accepts the factory itself.

`setTreasury` rejects only the zero address, and the factory has a payable `receive`, so a launch fee paid to a self set treasury succeeds while the ether never leaves. The hook already guards this case with `SelfPayment`, and the factory does not.

[https://github.com/SB-Security/audit-2026-07-](https://github.com/SB-Security/audit-2026-07-CashCat/blob/77d08fa93fb6b39432446290c65f8ef47ab4b489/src/v4/CashCatFactoryVNext.sol#L1161-L1165)

[CashCat/blob/77d08fa93fb6b39432446290c65f8ef47ab4b489/src/v4/CashCatFactoryVNext.sol#L1161-L1165](https://github.com/SB-Security/audit-2026-07-CashCat/blob/77d08fa93fb6b39432446290c65f8ef47ab4b489/src/v4/CashCatFactoryVNext.sol#L1161-L1165)

6. A treasury that cannot receive native value blocks platform collection.

`collectPlatform` clears the native platform tab and then pushes to the current treasury, so a destination that rejects ETH reverts the call and restores the tab. The balance stays accounted for and only `setTreasury` clears the condition.

L-09 · CONTINUED

<https://github.com/SB-Security/audit-2026-07-CashCat/blob/77d08fa93fb6b39432446290c65f8ef47ab4b489/src/v4/CashCatHookV2.sol#L583>

7. A self burner is admitted on three getters and no generation marker.

`publishModuleSet` identifies a self burner by its factory, hook and pool manager only. The token master is identified positively through `GENERATION` and `PIPS_PER_BP`, and the burner slot never receives the same treatment, so a legacy burner that answers the three getters is publishable and fails later at registration.

<https://github.com/SB-Security/audit-2026-07-CashCat/blob/77d08fa93fb6b39432446290c65f8ef47ab4b489/src/v4/CashCatFactoryVNext.sol#L943-L945>

8. Converter routes are keyed by quote token alone.

A route stores one fee source under one token key, so publishing a route for a second live hook generation overwrites the first and leaves its booked revenue outside the permissionless conversion path.

<https://github.com/SB-Security/audit-2026-07-CashCat/blob/77d08fa93fb6b39432446290c65f8ef47ab4b489/src/v4/CashCatRevenueConverter.sol#L234>

9. Fee routes accept recipients that cannot exercise the rights granted.

`_wireFeeRoute` accepts any nonzero address for a single recipient route and rejects only the splitter itself for a multi recipient route, so a launcher can name a protocol contract whose share nothing can later claim or rotate.

<https://github.com/SB-Security/audit-2026-07-CashCat/blob/77d08fa93fb6b39432446290c65f8ef47ab4b489/src/v4/CashCatFactoryVNext.sol#L621>

IMPACT

No funds are at risk and no unprivileged caller can reach any of these.

Item 1 is the only one that works and then stops working: a route validates, converts normally, and later reverts every conversion once the ring wraps past the configured window. Revenue stays booked and the owner can reinstall a shorter window, so it is recoverable, but nothing signals the condition before it bites.

Items 3, 7 and 8 publish cleanly and then fail at first use, spending a creator's launch transaction on an error that does not name its cause. Items 2, 5, 6 and 9 park or strand value, recoverable by the owner in every case except 9. Item 4 is a divergence between the specification and the code rather than a defect in either.

Grouped rather than filed separately because each is a single missing check and the aggregate is a pattern worth addressing once: the factory states the invariant that an enabled config can never be born unlaunchable, and these are the places that invariant, and its equivalents elsewhere, are not enforced.

RECOMMENDATION

Validate each value where it is set.

Bound the accepted TWAP window by the history the ring can actually guarantee, deriving the ceiling from `type(uint16).max` and the chain's block time rather than from a fixed 86,400, and reject a window that `_cardinalityFor` would have to clamp. Require the settled token amount to equal the minted supply in `unlockCallback` and revert the launch when exact full supply seeding cannot be represented. Compute `tickSpacingToMaxLiquidityPerTick` in `publishConfig` and reject seed liquidity above it at either range boundary. Check `feeRate` against an explicit allowlist of the four documented values, or revise the documentation to describe the range actually permitted. Reject `address(this)` in the factory's treasury setter, matching the hook. Give the treasury a pull based withdrawal, or leave a failed push non blocking with a recoverable credit. Require a generation constant and a V2 only selector on the self burner slot, as the token master already requires. Key converter routes by fee source and quote token together. Reject known protocol addresses when wiring a fee route, or require the recipient to accept through an explicit callback before the launch finalizes.

L-10

LOW

● FIXED

The converter static per-call ceiling permanently strands a route platform fees, if a thinly-traded quote is ever approved

`CashCatRevenueConverter` sizes every sale from a figure fixed at configuration time, validated only as non-zero:

`CashCatRevenueConverter.sol`

```
if (maxPerCall == 0) revert InvalidCap();
```

The only depth check anywhere in `setRoute` is that the pool is not completely empty:

`CashCatRevenueConverter.sol`

```
if (pool.liquidity() == 0) revert PoolHasNoLiquidity();
```

Nothing relates the ceiling to the pool it sells into. That matters because `convert` prices a floor from the pre-trade averages and then rejects the result against it, so a sale must fit its own impact inside the band:

`CashCatRevenueConverter.sol`

```
uint256 floor = _floorFor(r, amountIn);
...
if (ethOut < floor) revert SlippageExceeded(ethOut, floor);
```

And the sale size is not the tab, it is the tab clamped to the ceiling:

`CashCatRevenueConverter.sol`

```
uint256 booked = IFeeSource(r.feeSource).platformTab(token);
return booked < r.maxPerCall ? booked : r.maxPerCall;
```

Below the ceiling the sale shrinks with the tab and clears. At or above it, `amountIn` pins at `maxPerCall` and stays there. If that figure is too large for the pool, every `convert` reverts and because the tab only ever grows it never falls back into the working range.

The precondition is a listing decision. `setApprovedQuote` gates a quote on being a contract and nothing else:

`CashCatFactoryVNext.sol`

```
function setApprovedQuote(address quote, bool approved) external onlyOwner {
    if (quote == address(0)) revert InvalidConfig();
    if (approved && quote.code.length == 0) revert NotAContract();
    approvedQuote[quote] = approved;
    emit QuoteApprovalUpdated(quote, approved);
}
```

The TWAP floor does not protect against this. It is what rejects the sale, so it is the mechanism that makes the stall permanent rather than the thing preventing it.

IMPACT

No impact on anything approved today. USDG's pool is deep enough that the canonical sale uses about a quarter of its band.

If a thinly-traded quote is listed and its route is configured with parameters carried over from USDG's, that route's platform fees become permanently unconvertible. Nothing is lost, the balance stays booked in `platformTab` on the hook and the ledger still agrees with it and the owner can recover by lowering `maxPerCall` through `setRoute`. But nothing surfaces the condition: a failed `convert` reverts with no event and the route is indistinguishable from an idle one from outside.

L-10 · CONTINUED

The failure also hides while it is cheap to fix. It only appears once fees have accumulated past the ceiling, which is when there is the most stuck.

RECOMMENDATION

Bound the sale by live depth rather than by a figure measured once. The pattern is already in the codebase, on the burner, which recomputes against the pool on every call:

[CashCatSelfBurnerV2.sol#L351-L361](#)

Applied here, `convertibleNow` becomes `min(platformTab, maxPerCall, depthCap)`. The sale then shrinks as liquidity falls, keeps clearing its own floor, drains the tab over more calls instead of stopping and recovers on its own when depth returns.

A tighter check in `setRoute` is not a substitute. The route is configured once and the depth moves afterwards, so a snapshot assertion would pass and then go stale.

If the conversion path is not to be touched, a serviceability view, whether a sale of `convertibleNow()` would currently clear its floor, at least turns a silent permanent stall into something a monitor catches the first time it happens.



AUDIT COMPLETE

Stay secure.

Thank you for choosing SBSecurity.

WEBSITE

sbsecurity.net

TELEGRAM

[@blckhv](https://t.me/blckhv)

REPORT

[letscash](#) · August 2026